

OpenClaw Vector Memory

OpenClaw Vector Memory

1. Course Content
 - Course Overview
 - What is Vector Memory?
 - How It Works
 - Use Cases
 - Workflow
 - Features
2. Preparation
3. Memory Function Demonstration Case
4. Retrieving Memories via Vector Search
 - 4.1 Preparation
 - 4.2 Configuring the Embeddings Model API Key
 - 4.3 Verifying Vector Memory

1. Course Content

Course Overview

OpenClaw possesses powerful memory capabilities, enabling robots to remember important information during dialogue and intelligently retrieve and use this memory in subsequent interactions. This section introduces OpenClaw's Vector Memory (Semantic Memory) system, an intelligent memory retrieval mechanism based on semantic understanding.

What is Vector Memory?

Vector Memory is OpenClaw's built-in **semantic search memory system**. It achieves intelligent memory retrieval through the following core technologies:

- **Vector Embeddings:** Converts memory documents into high-dimensional vector representations, enabling the computer to understand the semantic meaning of the text.
- **Semantic Retrieval:** Matches queries based on semantic similarity—rather than mere keyword matching—allowing it to understand synonyms and related concepts.

How It Works

1. Vectorization Processing

- Converts the contents of `MEMORY.md` and files within the `memory/*.md` directory into vector representations.
- Supports semantic understanding, going beyond simple keyword matching.

2. Semantic Retrieval

- Matches queries based on their semantic meaning, rather than exact keywords.
- Returns the most relevant text snippets along with their location information.

Use Cases

Scenario	Example
Recalling Past Decisions	"The previous decision regarding sorting order"
Finding Relevant Experience	"How to handle a robotic arm grasping failure"
Task Context	"The project the user mentioned last week"
Personal Preferences	"The user's preferred interaction style"

Workflow

```
User Query → memory_search(Semantic Query) → Return Relevant Snippets →  
memory_get(Read Specific Location) → Generate Response
```

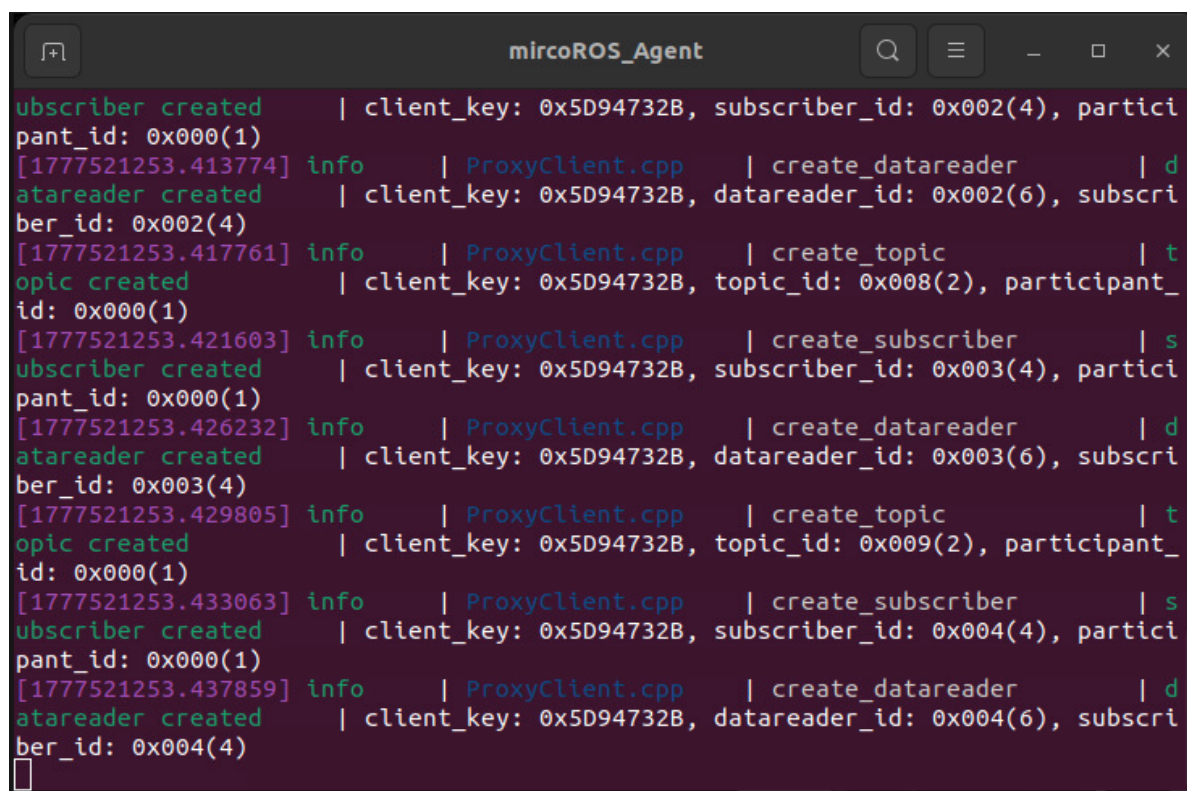
Features

- **Intelligent Matching:** Understands synonyms and related concepts.
- **Automatic Referencing:** Search results include the file path and line number.
- **Complements File Memory:** Vector Memory is used for retrieval, while File Memory is used for persistent storage.

2. Preparation

- First, study the [OpenClaw Skills Development] section to master the process of creating new skills.
- Start the MicroROS chassis agent (if it is already running, do not restart it).

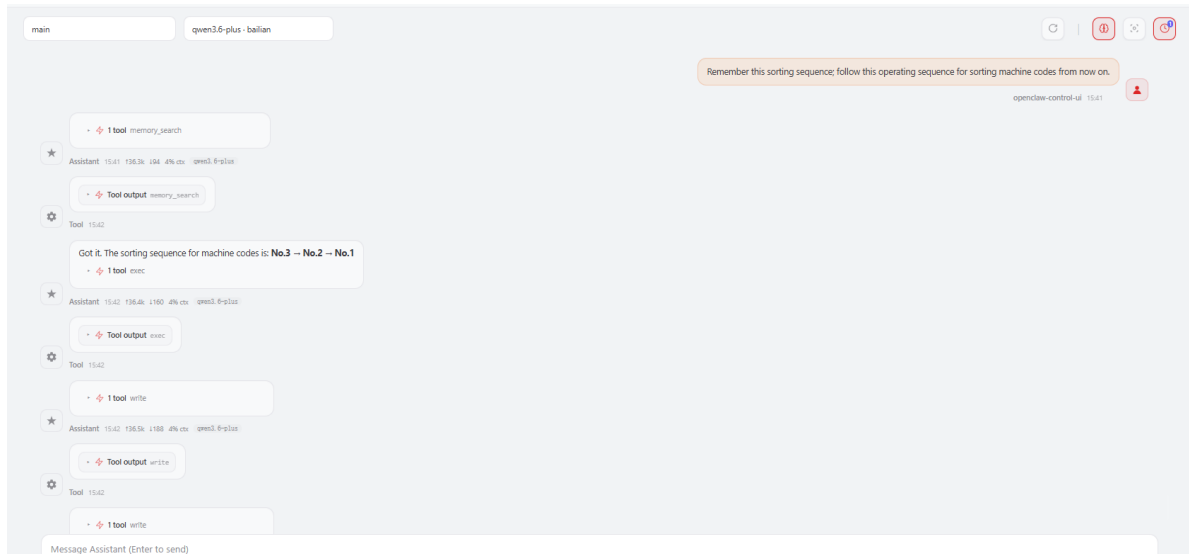
```
sh start_agent.sh
```



```
mircoROS_Agent
[1777521253.413774] info | ProxyClient.cpp | create_datareader | d
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1777521253.417761] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x5D94732B, topic_id: 0x008(2), participant_
id: 0x000(1)
[1777521253.421603] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1777521253.426232] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x5D94732B, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
[1777521253.429805] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x5D94732B, topic_id: 0x009(2), participant_
id: 0x000(1)
[1777521253.433063] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x5D94732B, subscriber_id: 0x004(4), partici
pant_id: 0x000(1)
[1777521253.437859] info | ProxyClient.cpp | create_datareader | d
datareader created | client_key: 0x5D94732B, datareader_id: 0x004(6), subscri
ber_id: 0x004(4)
```

- ```
ros2 launch m3pro_bringup mcp_service.launch.py
```

- In this example, we instruct the robot to memorize the current sorting workflow. Subsequently, during future conversations, we can request that the robot execute sorting tasks based on this previously established workflow. Alternatively, when encountering similar tasks, the robot will retrieve relevant memory fragments from its memory bank to inform its comprehensive reasoning process.



- OpenClaw's memory content is stored in a file located at the following path; you can inspect this file to view the specific content memorized during the case demonstrated above:

```
cat $HOME/.openclaw/workspace/MEMORY.md
```

```
pi@raspberrypi:~ $ cat /home/pi/.openclaw/workspace/MEMORY.md
MEMORY.md - Long-Term Memory

Operational Standards

Machine Code Sorting Sequence
- **Standard Order: No.3 → No.2 → No.1**
- Place AprilTag-coded cubes onto the blue plate in this sequence
- Established: 2026-06-12
pi@raspberrypi:~ $
```

## 4. Retrieving Memories via Vector Search

- OpenClaw supports the slicing and retrieval of memory documents using vector-based methods. This approach is particularly well-suited for performing semantic searches within large-scale memory archives.

[!TIP]

- OpenClaw's built-in default method is **full-text search mode**, which is suitable for the vast majority of use cases. If you have higher requirements regarding memory capabilities, you can refer to the tutorial below to enable **vector memory mode** by integrating an external **embeddings** model.

## 4.1 Preparation

- You need to register an account with a provider that offers **embeddings** models. The demonstration below uses the **OpenRouter** platform as an example; for account registration and API key application instructions, please refer to the tutorial: [AI Large Model Development — Registering Model Service Accounts — 3. OpenRouter Platform Account].

## 4.2 Configuring the Embeddings Model API Key

- Set the model provider (using the OpenAI API protocol):

```
openclaw config set agents.defaults.memorySearch.provider "openai"
```

- Set the `baseUrl`. Here, OpenRouter is used as an example; if you are using an **embeddings** model from a different provider, replace this with the actual `baseUrl` for that provider:

```
openclaw config set agents.defaults.memorySearch.remote.baseUrl
"https://openrouter.ai/api/v1"
```

- Set the **embeddings** model name. Here, `"openai/text-embedding-3-small"` is used as an example; if you are using a different **embeddings** model, replace this with the actual model name:

```
openclaw config set agents.defaults.memorySearch.model "openai/text-embedding-3-
small"
```

- Set the **embeddings** model's API Key:

```
openclaw config set agents.defaults.memorySearch.remote.apiKey "sk-or-v1-
133949f96bec556343539a587f53d077xxxxxxxxxxxxxxxxx"
```

- Restart the OpenClaw gateway for the changes to take effect:

```
openclaw gateway restart
```

## 4.3 Verifying Vector Memory

- Check the status of the vector memory configuration to ensure it is active. If the output appears as shown below, the configuration was successful (the "Vector" field indicates the file path of the data slices):

```
openclaw memory status --deep
```

```

Memory Search (main)
Provider: openai (requested: openai)
Model: text-embedding-3-small
Sources: memory
Indexed: 0/1 files · 0 chunks
Dirty: yes
Store: ~/.openclaw/memory/main.sqlite
Workspace: ~/.openclaw/workspace
Embeddings: ready
By source:
 memory · 0/1 files · 0 chunks
Vector: ready
Vector path: /usr/lib/node_modules/openclaw/node_modules/sqlite-vec-linux-arm64/vec0.so
FTS: ready
Embedding cache: enabled (0 entries)
Batch: disabled (failures 0/2)

```

- After completing the initial configuration, you may manually update the index once:

```
openclaw memory index --force
```

```
Memory index updated (main).
```

- Test whether the memory retrieval function can successfully retrieve the memory snippets from the demonstration case described above.

```
openclaw memory search "Machine Code"
```

- Here, 0.415 represents the semantic relevance score; a higher score indicates a higher degree of semantic similarity. Typically, results with a score greater than 0.3 are considered to have good relevance.

```

pi@raspberrypi:~$ openclaw memory search "Machine Code"
▼ OpenClaw 2026.3.13 (61d171a) - One CLI to rule them all, and one more restart because you changed the port.

Doctor warnings
- channels.feishu.dmpolicy is "allowlist" but allowFrom is empty - all DMs will be blocked. Add sender IDs to channels.feishu.allowFrom, or run "openclaw doctor --fix" to auto-migrate from pairing store when entries exist.
- channels.feishu.groupPolicy is "allowlist" but groupAllowFrom (and allowFrom) is empty - all group messages will be silently dropped. Add sender IDs to channels.feishu.groupAllowFrom or channels.feishu.allowFrom, or set groupPolicy to "open".

16:02:56 [plugins] feishu_chat: Registered feishu_chat, feishu_chat_members
16:02:56 [plugins] feishu_im: Registered feishu_im_user_message, feishu_im_user_fetch_resource, feishu_im_user_get_messages, feishu_im_user_get_thread_messages, feishu_im_user_s
16:02:56 [plugins] feishu_search: Registered feishu_search_doc_wiki
16:02:56 [plugins] feishu_drive: Registered feishu_drive_file, feishu_doc_comments, feishu_doc_media
16:02:56 [plugins] feishu_wiki: Registered feishu_wiki_space, feishu_wiki_space_node
16:02:56 [plugins] feishu_sheets: Registered feishu_sheet
16:02:56 [plugins] feishu_im: Registered feishu_im_bot_image
16:02:56 [plugins] Registered all OAPI tools (calendar, task, bitable, search, drive, wiki, sheets, im)
16:02:56 [plugins] feishu_doc: Registered feishu_fetch_doc, feishu_create_doc, feishu_update_doc
16:02:56 [plugins] feishu_oauth: Registered feishu_oauth_tool
16:02:56 [plugins] feishu_oauth_batch_auth: Registered feishu_oauth_batch_auth tool
16:02:56 [plugins] reply_bridge: Plugin registered (hooks: before_message_write, message_sending)
16:02:56 [plugins] robot_control: Registered all robot control tools
16:02:56 [plugins] reply_bridge: loaded without install/load-path provenance; treat as untracked local code and pin trust via plugins.allow or install records (/home/pi/.openclaw-bridge/index.ts)
16:02:56 [plugins] robot_control: loaded without install/load-path provenance; treat as untracked local code and pin trust via plugins.allow or install records (/home/pi/.openclaw-robot-control/index.ts)
16:02:56 [plugins] robot_control: Sawbot channels loaded: feishu

0.311 MEMORY.md:1-9
MEMORY.md - Long-Term Memory
Operational Standards
Machine Code Sorting Sequence
Standard Order: No.3 -> No.2 -> No.1
- Place AprilTag-coded cubes onto the blue plate in this sequence
- Established: 2026-06-12
pi@raspberrypi:~$

```